

Algoritmica Grafurilor - Cursul 3

16 ottobre 2020

- 1 **Vocabularul teoriei grafurilor**
 - Matrici asociate
- 2 **Structuri de date**
- 3 **Probleme de drumuri în (di)grafuri**
 - Traversarea (di)grafurilor
 - Drumuri de cost minim
- 4 **Exerciții pentru seminarul de săptămâna viitoare**
- 5 **Exerciții rezolvate (parțial)**

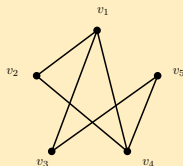
Matrici asociate - Matricea de adiacență

Fie G un graf cu $V(G) = \{v_1, \dots, v_n\}$. **Matricea de adiacență** a lui G este matricea $A = (a_{ij})_{1 \leq i, j \leq n} \in \mathcal{M}_{n \times n}(\{0, 1\})$, unde

$$a_{ij} = \begin{cases} 1, & \text{dacă } v_i \text{ și } v_j \text{ adiacente} \\ 0, & \text{altfel} \end{cases}.$$

Exemplu

Un graf și matricea sa de adiacență.



$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

Matrici asociate - Matricea de incidență

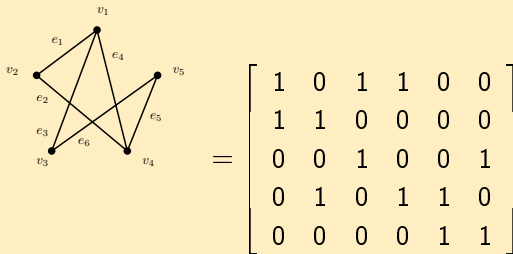
Fie G un graf cu $V(G) = \{v_1, \dots, v_n\}$ și $E(G) = \{e_1, \dots, e_m\}$.

Matricea de incidență a lui G este matricea $B = (b_{ij})_{1 \leq i, j \leq n} \in \mathcal{M}_{n \times m}(\{0, 1\})$, unde

$$b_{ij} = \begin{cases} 1, & \text{dacă } e_j \text{ este incidentă cu } v_i \\ 0, & \text{altfel} \end{cases}.$$

Exemplu

Un graf și matricea sa de incidență.



Valorile proprii, vectorii proprii și polinomul caracteristic ale matricei de adiacență sunt numite **valorile proprii**, **vectorii proprii**, și, respectiv, **polinomul caracteristic al grafului** corespunzător. Acestea sunt obiectele de studiu ale **teoriei spectrale a grafurilor**.

Pentru digrafuri, matrici similare pot fi definite cu elemente din $\{-1, 0, 1\}$ pentru a marca direcția arcelor.

Fie G un digraf cu $V(G) = \{v_1, \dots, v_n\}$ și $E(G) = \{e_1, \dots, e_m\}$. **Matricea de incidență nod-arc** a lui G este matricea $B = (b_{ij})_{1 \leq i, j \leq n} \in \mathcal{M}_{n \times m}(\{-1, 0, 1\})$, unde

$$b_{ij} = \begin{cases} 1, & \text{dacă } e_j \text{ este incident dinspre } v_i \\ -1, & \text{dacă } e_j \text{ este incident către } v_i \\ 0, & \text{altfel} \end{cases}.$$

Structuri de date pentru matricea de adiacență

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Fie $G = (V, E)$ un (di)graf cu $V = \{1, 2, \dots, n\}$.

- Dacă $A = (a_{ij})_{1 \leq i, j \leq n}$ este matricea de adiacență a lui G atunci, reprezentând-o ca un tablou 2-dimensional, avem nevoie de $\mathcal{O}(n^2)$ timp pentru inițializare (în funcție de limbajul de programare).
- Astfel, orice algoritm care reprezintă G cu matrice de adiacență are complexitatea timp (și spațiu) $\Omega(n^2)$.
- Testarea dacă două noduri fixate sunt adiacente se face în $\mathcal{O}(1)$, dar parcurgerea întregii mulțimi de vecini $N_G(u)$ (sau $N_{G+(u)}$), pentru un anume nod $u \in V$, necesită $\Omega(n)$ - prea mult pentru grafuri rare de ordin mare.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Structuri de date pentru listele de adiacență

Fie $G = (V, E)$ un (di)graf cu $V = \{1, 2, \dots, n\}$ și $|E| = e$.

- Orice nod $u \in V$ are o listă, $A(u)$, a vecinilor săi din G :
 - ▶ când G este graf $A(u) = N_G(u)$;
 - ▶ dacă G este digraf, atunci $A(u) = N_G^+(u) = \{v \in V : uv \in E\}$;
- Dacă G este graf, atunci fiecare muchie $uv \in E$ generează două elemente în listele de adiacență: unul în $A(u)$ și unul în $A(v)$; spațiul necesar este $\mathcal{O}(n + 2e)$.
- Dacă G este digraf, atunci spațiul necesar este $\mathcal{O}(n + e)$.
- Listele de adiacență pot fi implementate folosind liste înlanțuite sau tablouri.
- Testarea dacă un nod u este adiacent cu un altul v în G necesită $\Omega(d_G(u))$ timp, dar parcurgerea întregii mulțimi de vecini $N_G(u)$ (sau $N_G^+(u)$), pentru un nod $u \in V$, se poate face în $\Omega(d_G(u))$ (și nu în $\mathcal{O}(n)$ ca în cazul matricei de adiacență).

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Traversarea (sistematică a) grafurilor (graph search, graph traversal) este o paradigmă algoritmică care specifică o metodă sistematică de a parcurge mulțimea nodurilor care pot fi accesate (atinse) pe drumuri plecând dintr-un nod fixat al (di)grafului.

Dat un (di)graf $G = (\{1, \dots, n\}, E)$ și $s \in V(G)$

generează "eficient" mulțimea

$$S = \{u \in V(G) : \text{există un drum de la } s \text{ la } u \text{ în } G\}.$$

G va fi reprezentat cu liste de adiacență, deoarece, în timpul procesului de traversare, trebuie manipulată în mod eficient mulțimea de vecini a nodului curent.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Probleme de drumuri - Breadth-First Search (BFS)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

```
for  $v \in V$  do
     $label(u) \leftarrow -1$ ;  $parent(u) \leftarrow -1$ ;
 $label(s) \leftarrow 0$ ;  $parent(s) \leftarrow 0$ ;
crează o coadă  $Q$  conținând  $s$ ;
while  $Q \neq \emptyset$  do
     $u \leftarrow pop(Q)$ ;
    for  $v \in A(u)$  do
        if  $label(v) < 0$  then
             $label(v) \leftarrow label(u) + 1$ ;
             $parent(v) \leftarrow u$ ;  $push(Q, v)$ ;
```

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Proprietăți ale BFS. Nu este dificil de arătat că:

- $S = \{u \in V : \text{label}(u) \geq 0\}$;
- $\forall u \in V, \text{label}(u) = d_G(s, u)$ (distanța în G de la s la u);
- Variabila **parent** definește **arborele bfs** asociat parcurgerii din s : dacă G este graf atunci arborele bfs este un arbore parțial al componentei conexe conținând s ; dacă G este digraf atunci arborele bfs este o **arborescență** (arbore cu rădăcină, orientat - arcele sunt îndreptate către exteriorul rădăcinii s).
- Complexitatea timp a $BFS(s)$ este $\mathcal{O}(n_S + m_S)$, unde $n_S = |S| \leq |V| = n$, și $m_S = |E([S]_G)| \leq |E|$ (se observă că fiecare vecin din lista de adiacență a unui nod din S este accesat o singură dată).

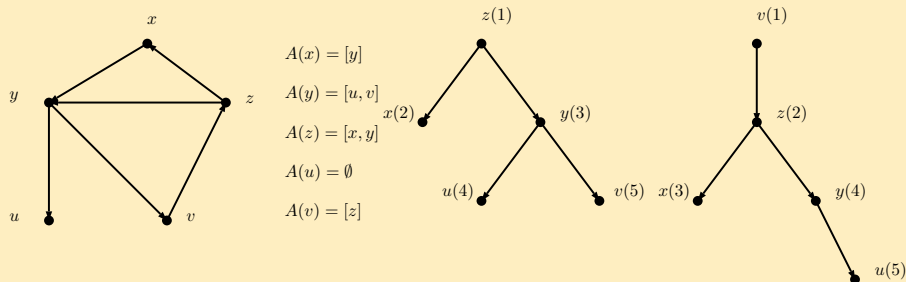
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Probleme de drumuri - Breadth-First Search (BFS)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exemplu

Două parcurgeri BFS ale unui aceluiași digraf (începând din z și din v):



(Am marcat în paranteze ordinea de vizitare.)

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Probleme de drumuri - Depth-First Search (DFS)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

```
for  $v \in V$  do
     $label(u) \leftarrow -1$ ;  $parent(u) \leftarrow -1$ ;
 $label(s) \leftarrow 0$ ;  $parent(s) \leftarrow 0$ ;
crează o stivă  $S$  conținând  $s$ ;  $n_S \leftarrow 0$ ;
while  $S \neq \emptyset$  do
     $u \leftarrow top(S)$ ;
    if  $((v \leftarrow next[A(u)]) \neq NULL)$  then
        if  $label(v) < 0$  then
             $n_S ++$ ;  $label(v) \leftarrow n_S$ ;
             $parent(v) \leftarrow u$ ;  $push(S, v)$ ;
        else
             $delete(S, u)$ ;
```

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Proprietăți ale DFS. Nu este dificil de arătat că:

- $\{u \in V : \text{label}(u) \geq 0\}$ este exact mulțimea S de noduri accesibile pe drumuri din s ;
- $\forall u \in V, \text{label}(u) = \text{momentul vizitării lui } u$ (s este vizitat la momentul 0);
- Variabila **parent** definește **arborele dfs** asociat parcurgerii din s
- Complexitatea timp a $DFS(s)$ este $\mathcal{O}(n_S + m_S)$, unde $n_S = |S| \leq |V| = n$, și $m_S = |E([S]_G)| \leq |E|$ (se observă că fiecare vecin din lista de adiacență a unui nod din S este accesat o singură dată).

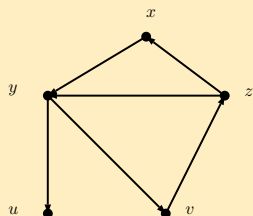
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Probleme de drumuri - Depth-First Search (DFS)

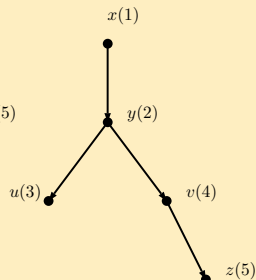
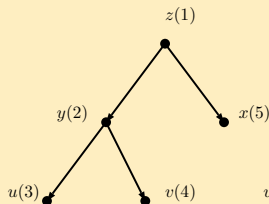
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exemplu

Două parcurgeri DFS ale unui aceluiași digraf (începând din z și din x):



$A(x) = [y]$
 $A(y) = [u, v]$
 $A(z) = [x, y]$
 $A(u) = \emptyset$
 $A(v) = [z]$



Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Fie $G = (V, E)$ un digraf, cu $V = \{1, \dots, n\}$.

- Fiecare arc (muchie orientată) $e \in E$ are asociat un cost $a(e) \in \mathbb{R}$ (pondere, lungime etc).
- Dacă G este reprezentat cu liste de adiacență, atunci $a(ij)$ este un câmp al elementului din lista de adiacență a lui i (reprezentând un arc ij).
- Pentru ușurința notației vom folosi reprezentarea lui G cu o matrice de cost-adiacență $A = (a_{ij})_{1 \leq i, j \leq n}$, unde

$$a_{ij} = \begin{cases} a(ij), & \text{dacă } ij \in E \\ \infty, & \text{altfel} \end{cases}.$$

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

- Aici ∞ reprezintă un număr real foarte mare relativ la costurile muchiilor (e.g., $\infty > n \cdot \max_{ij \in E} a(ij)$) și presupunem că $\infty \pm a = \infty$, $\infty + \infty = \infty$.
- Este de asemenea posibil să folosim ∞ ca un acces nereușit la structura de date utilizată pentru reprezentarea matricei A .
- Pentru $i, j \in V$, mulțimea tuturor drumurilor din G de la i la j este notată cu $\mathcal{P}_{ij}$:

$$\mathcal{P}_{ij} = \{P : P \text{ este un drum de la } i \text{ la } j\}.$$

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

- Dacă $P_{ij} \in \mathcal{P}_{ij}$, $P : (i =) v_0, v_0 v_1, v_1, \dots, v_{r-1}, v_{r-1} v_r, v_r (= j)$, atunci

$$V(P_{ij}) = \{v_0, v_1, \dots, v_r\}, E(P_{ij}) = \{v_0 v_1, \dots, v_{r-1} v_r\}.$$

- Costul lui $P_{ij} \in \mathcal{P}_{ij}$ este

$$a(P_{ij}) = 0 + \sum_{uv \in E(P_{ij})} a_{uv}.$$

- În particular $a(P_{ii}) = 0$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Principalele probleme de drum de cost minim

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Drumul de cost minim între două noduri.

P1 Date $G = (V, E)$ digraf; $a : E \rightarrow \mathbb{R}$; $s, t \in V, s \neq t$.

Găsiți $P_{st}^* \in \mathcal{P}_{st}$, așa încât $a(P_{st}^*) = \min \{a(P_{st}) : P_{st} \in \mathcal{P}_{st}\}$.

Drumurile de cost minim dintre un nod și toate celelalte noduri

P2 Date $G = (V, E)$ digraf; $a : E \rightarrow \mathbb{R}$; $s \in V$.

Găsiți $P_{si}^* \in \mathcal{P}_{si}, \forall i \in V$, a. î. $a(P_{si}^*) = \min \{a(P_{si}) : P_{si} \in \mathcal{P}_{si}\}$

Toate drumurile de cost minim.

P3 Date $G = (V, E)$ digraf; $a : E \rightarrow \mathbb{R}$.

Găsiți $P_{ij}^* \in \mathcal{P}_{ij}, \forall i, j \in V$, a. î. $a(P_{ij}^*) = \min \{a(P_{ij}) : P_{ij} \in \mathcal{P}_{ij}\}$

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Remarci 1

- Reprezentarea cu matrice de cost-adiacență a perechii (G, a) implică $P_{ij} \neq \emptyset, \forall i, j \in V(G)$: dacă $a(P_{ij}) < \infty$, atunci P_{ij} este un drum real în G , iar dacă $a(P_{ij}) = \infty$, atunci P_{ij} nu este un drum în G dar este un drum în digraful complet simetric obținut din G prin adăugarea tuturor arcelor lipsă (cu costuri ∞).
- Urmează că toate mulțimile din care se determină un element (drum) de cost minim în problemele **P1** - **P3** sunt nevide și finite și **drumurile minime cerute sunt bine definite**.
- Algoritmii pentru rezolvarea problemei **P1** se pot obține din cei pentru rezolvarea problemei **P2** prin adăugarea unei condiții (evidente) de oprire.
- Problema **P3** poate fi rezolvată prin iterarea oricărui algoritm pentru problema **P2**. Vom vedea că sunt și soluții mai eficiente.

1. Rețele de comunicare (Communication Networks). Digraful $G = (V, E)$ reprezintă o rețea de comunicații între nodurile din V , iar E modelează mulțimea **legăturilor orientate** dintre noduri.

- Dacă $a(e) \geq 0$ ($\forall e \in E$) reprezintă lungimea conexiunii directe dintre extremitățile lui e , atunci problemele P1 - P3 sunt **probleme de drumuri de cost minim** naturale.
- Dacă $a(e) \geq 0$ ($\forall e \in E$) reprezintă timpul necesar pentru parcurgerea conexiunii directe dintre extremitățile lui e , atunci problemele P1 - P3 **probleme pentru determinarea drumurilor celor mai rapide**.
- Dacă $a(e) \in (0, 1]$ ($\forall e \in E$) reprezintă probabilitatea funcționării corecte a conexiunii directe dintre extremitățile lui e și dacă presupunem că muchiile funcționează **independent** una de cealaltă, atunci problemele P1 - P3 sunt **probleme pentru determinarea drumurilor celor mai sigure**:

Dacă $P_{ij} \in \mathcal{P}_{ij}$ pentru o pereche $i, j \in V$, atunci **probabilitatea** ca acest drum să funcționeze corect este (datorită independenței)

$$Prob(P_{ij}) = \prod_{e \in E(P_{ij})} a(e).$$

Luând $a'(e) = -\log a(e)$,

$$\log Prob(P_{ij}) = \log \left(\prod_{e \in E(P_{ij})} a(e) \right) = - \sum_{e \in E(P_{ij})} a'(e).$$

Funcția $\log$ fiind monotonă, problemele P1 - P3 cu costurile a' , oferă **drumurile cele mai fiabile** în rețele de comunicare.

2. Rețele PERT - Metoda drumului critic (Critical path method - CPM). PERT (Path Evaluation and Review Technique) este o metodă de a analiza (îndeosebi) îndeplinirea fiecărei sarcini dintr-un proiect mai complex.

- Fie $P = \{A_1, A_2, \dots, A_n\}$ activitățile atomice ale unui mare proiect P (n este foarte mare). $(P, <)$ este o mulțime parțial ordonată, unde $A_i < A_j$ dacă $i \neq j$ și activitatea A_j poate începe doar după ce activitatea A_i s-a terminat.
- Pentru fiecare activitate A_i , timpul necesar finalizării t_i este cunoscut (sau doar estimat).

Găsiți o planificare a activităților acestui proiect care să minimizeze durata totală până la finalizare.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Asociem acestei problem un digraf aciclic astfel:

- pentru fiecare activitate A_p ($p \in \{1, \dots, n\}$) adăugăm un arc $i_p j_p$ de cost $a(i_p j_p) = t_p$;
- nodul i_p corespunde startului activității A_p iar nodul j_p este asociat finalizării ei;
- dacă activitatea A_k poate porni doar după activitatea A_p adăugăm arcul $j_p i_k$ (o activitate fictivă - dummy activity).

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Drumuri de cost minim - Aplicații

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

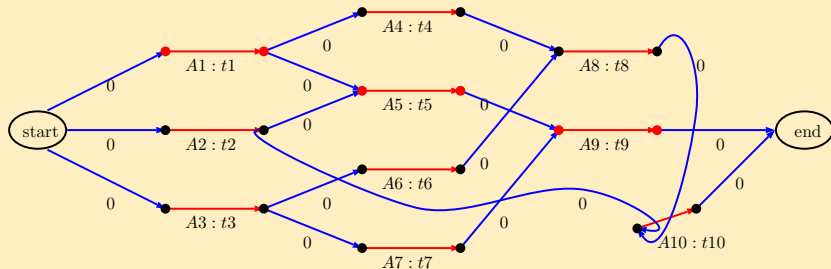
- Construcția digrafului este încheiată după adăugarea unui nod s corespunzând momentului inițial al proiectului, legat prin arce $s_i p$ pentru fiecare activitate A_p în care nu intră vreun arc și a unui nod t corespunzând finalizării proiectului, legat prin arce $j_p t$ pentru fiecare activitate A_p din care nu iese vreun arc.
- În digraful obținut **costul maxim al unui drum de la s la t este egal cu timpul minim necesar îndeplinirii în întregime a proiectului.**
- Un drum de cost maxim este numit a **drum critic** deoarece orice întârziere a unei activități de pe acest drum implică o întârziere a întregului proiect.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Drumuri de cost minim - Aplicații

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exemplu



C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

3. Problema rucsacului (0 – 1). Avem un rucsac de dimensiune $b \in \mathbb{N}$, și n obiecte de dimensiuni $a_1, \dots, a_n \in \mathbb{N}$. Se cunoaște și profitul $p_i \in \mathbb{N}$ al adăugării obiectului i ($i \in \{1, \dots, n\}$) în rucsac. Se cere **să se găsească o completare a rucsacului care să maximizeze profitul total.**

Fie X_i , pentru $i \in \{1, \dots, n\}$, o variabilă booleană cu semnificația $x_i = 1$ dacă și numai dacă obiectul i este pus în rucsac. Problema rucsacului poate fi descrisă astfel

$$\max \left\{ \sum_{i=1}^n p_i x_i : \sum_{i=1}^n a_i x_i \leq b, x_i \in \{0, 1\}, \forall i = \overline{1, n} \right\}.$$

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

- Fie $G = (V, E)$ un digraf cu $V = \{s\} \cup V_1 \cup \dots \cup V_n \cup \{t\}$, unde $V_i = \{i^0, i^1, \dots, i^b\}$ este asociat obiectului i , $i = \overline{1, n}$.
- Arcele lui G și costurile sunt:
 - ▶ $s1^0$ și $s1^{a_1}$ cu $a(s1^0) = 0$, $a(s1^{a_1}) = p_1$ (obiectul 1 este adăugat rucsacului cu profitul p_1 și nivelul de umplere a_1 , sau nu este adăugat, cu profitul și nivelul de umplere 0).
 - ▶ $(i-1)^j i^j$ cu $a((i-1)^j i^j) = 0$, $\forall i = \overline{2, n}, \forall j = \overline{0, b}$ (obiectul i nu este adăugat rucsacului: după competarea cu primele $i-1$ obiecte și nivelul de umplere j se trece la umplerea cu primele i obiecte, fără obiectul i ; nivelul de umplere rămâne neschimbat j iar profitul adăugat este 0).
 - ▶ Dacă $j - a_i \geq 0$, atunci avem și arcul $(i-1)^{j-a_i} i^j$ cu $a((i-1)^{j-a_i} i^j) = p_i$ (se poate ajunge la nivelul de umplere j prin adăugarea obiectului i la o umplere cu primele $i-1$ obiecte, cu nivelul de umplere $j - a_i$).

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

► $n^j t$ cu $a(n^j t) = 0, \forall j = \overline{0, b}$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

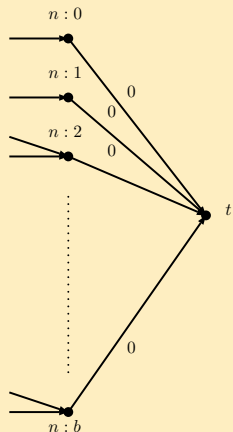
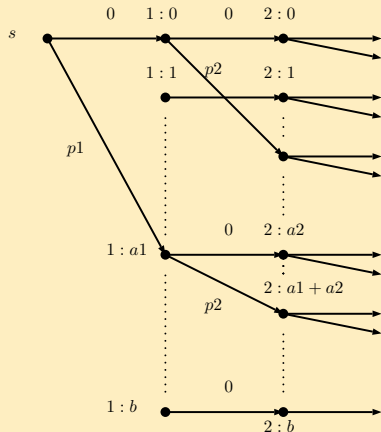
Remarci 2

- Fiecare drum de la s la t în G corespunde unei submulțimi de obiecte cu nivelul de umplere $\leq b$ și cu profitul total egal cu costul drumului. Deoarece, reciproc, fiecărei umpleri a rucsacului îi corespunde un drum de la s la t în G , urmează că problema rucsacului poate fi rezolvată prin determinarea unui drum de cost maxim în digraful aciclic G .
- Descrierea statică dată mai sus pentru G poate fi transformată într-una procedurală, folosind programarea dinamică. Problema aceasta este NP-hard (ordinul lui G poate fi exponențial în dimensiunea problemei).

Drumuri de cost minim - Aplicații

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Exemplu



- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

P2 Dat $G = (V, E)$ digraf; $a : E \rightarrow \mathbb{R}$; $s \in V$.

Găsiți $P_{si}^* \in \mathcal{P}_{si}, \forall i \in V$, a. î. $a(P_{si}^*) = \min \{a(P_{si}) : P_{si} \in \mathcal{P}_{si}\}$

Teorema 1

Fie G un digraf, $s \in V(G) = \{1, \dots, n\}$ și $a : E(G) \rightarrow \mathbb{R}$, a. î.

(I) $a(C) > 0$, pentru toate circuitele C din G .

Atunci $(u_1, \dots, u_n)$ este o soluție a sistemului de ecuații

$$(B) \quad \begin{cases} u_s = 0 \\ u_i = \min_{j \neq i} (u_j + a_{ji}) \end{cases} \text{ dacă și numai dacă}$$

$\forall i \in V(G), \exists P_{si}^* \in \mathcal{P}_{si}$ a. î. $u_i = a(P_{si}^*) = \min \{a(P) : P \in \mathcal{P}_{si}\}.$

Drumuri de cost minim - Rezolvarea problemei P2

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Proof:

" $\Leftarrow$ ". Fie P_{si}^* o soluție optimă a problemei P2 și $u_i = a(P_{si}^*)$.

Ipoteza (I) implică $u_s = 0$, i.e., prima ecuație a sistemului (B) este satisfăcută. Pentru $i \neq s$, drumul P_{si}^* are penultimul nod j . Dacă P_{sj} este drumul de la s la j determinat pe P_{si}^* de j , avem

$$u_i = a(P_{si}^*) = a(P_{sj}) + a_{ji} \geq a(P_{sj}^*) + a_{ji} = u_j + a_{ji}.$$

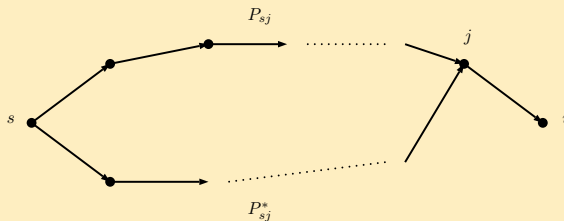
Arătăm că $u_i = u_j + a_{ji}$. Presupunem că $u_i > u_j + a_{ji}$, i. e., $a(P_{sj}) > a(P_{sj}^*)$.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Drumuri de cost minim - Rezolvarea problemei P2

Cazul 1. $i \notin V(P_{sj}^*)$. Atunci $P^1 = P_{sj}^* \circ (j, ji, i) \in \mathcal{P}_{si}$ și $a(P^1) = a(P_{sj}^*) + a_{ji} < a(P_{sj}) + a_{ji} = a(P_{si}^*)$, contradicție (P_{si}^* este un drum de cost minim).

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

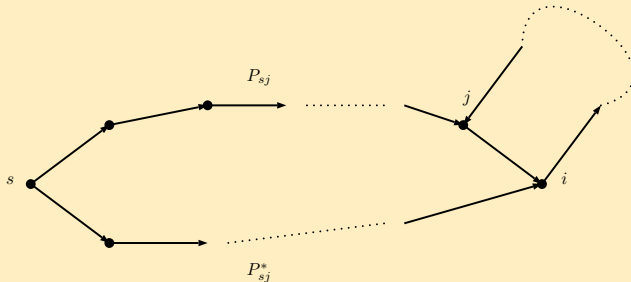


Drumuri de cost minim - Rezolvarea problemei P2

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Cazul 2. $i \in V(P_{sj}^*)$. Fie $P_{sj}^* = P_{si} \circ P_{ij}$ cele două drumuri determinate de nodul i on P_{sj}^* . Atunci costul circuitului $C = P_{ij} \circ (j, ji, i)$ este $a(C) = a(P_{ij}) + a_{ji} = a(P_{sj}^*) - a(P_{si}) + a_{ji} = u_j + a_{ji} - a(P_{si})$ care este $\leq u_j + a_{ji} - a(P_{si}^*) = u_j + a_{ji} - u_i < 0$, contradicție (ipoteza (I) este încălcată).

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.



Drumuri de cost minim - Rezolvarea problemei P2

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Astfel partea " $\Leftarrow$ " este demonstrată.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Remarcă 1

Am demonstrat mai sus că dacă j este nodul dinaintea lui i pe un drum de cost minim de la s la i , atunci drumul de la s la j determinat de j de pe acest drum este un drum de cost minim de la s la j . Inductiv, urmează:

Principiul de optimalitate al lui Bellman: Dacă P_{si}^* este un drum de cost minim de la s la i , atunci $\forall j \in V(P_{si}^*)$, dacă $P_{si}^* = P_{sj} \circ P_{ji}$, atunci P_{sj} (respectiv P_{ji}) este un drum de cost minim de la s la j (respectiv de la j la i).

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Drumuri de cost minim - Rezolvarea problemei P2

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

" $\Rightarrow$ ". Arătăm că dacă $(u_1, \dots, u_n)$ este o soluție a sistemului (B), atunci

(a) $\exists P_{si} \in \mathcal{P}_{si}$ așa încât $u_i = a(P_{si}), \forall i \in V$.

(b) $\forall i \in V, u_i = \min \{a(P) : P \in \mathcal{P}_{si}\} (= a(P_{si}))$.

(a) Dacă $i = s$, atunci $u_s = 0$ și drumul P_{ss} satisface $a(P_{ss}) = 0 = u_s$.
Dacă $i \neq s$, considerăm următorul algoritm

$v \leftarrow i; k \leftarrow 0;$

while $v \neq s$ **do**

 find w a. î. $u_v = u_w + a_{vw}; // \exists w$ deoarece u_v satisface (B)

$i_k \leftarrow v; k++; v \leftarrow w;$

Algoritmul determină drumul $P : (s =) i_{k+1}, i_{k+1} i_k, i_k, \dots, i_1, i_1 i_0, i_0 (= i)$ cu $P \in \mathcal{P}_{si}$ și

- Graph Algorithms - C. Croitoru - Graph Algorithms - C. Croitoru - Graph Algorithms

Drumuri de cost minim - Rezolvarea problemei P2

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

$$\begin{aligned}a(P) &= a(i_{k+1} i_k) + \cdots + a(i_1 i_0) = \\&= (u_{i_k} - u_{i_{k+1}}) + (u_{i_{k-1}} - u_{i_k}) + \cdots + (u_{i_0} - u_{i_1}) = \\&= u_{i_0} - u_{i_{k+1}} = u_i - u_s = u_i,\end{aligned}$$

În fiecare iterație **while** $w \notin \{i_0, \dots, i_{k-i}\}$ (altfel obținem un circuit de cost 0, în contradicție cu ipoteza (I)).

Cu notațiile din algoritmul de mai sus avem $u_i = u_{i_1} + a_{i_1 i}$.

(b) Fie $\bar{u}_i = a(P_{s i}^*)$, $\forall i \in V$. Din demonstrația anterioară, $\bar{u}_i$, $i = \overline{1, n}$ satisface sistemul (B).

Presupunem că $u = (u_1, \dots, u_n) \neq (\bar{u}_1, \dots, \bar{u}_n) = \bar{u}$.

Deoarece $u_s = \bar{u}_s = 0$, urmează că există un $i \neq s$ așa încât $u_i \neq \bar{u}_i$ și $\forall j \in V(P_{s i})$. $j \neq i$, $u_j = \bar{u}_j$, unde $P_{s i}$ este drumul construit la (a) pentru $\bar{u}_i$.

Drumuri de cost minim - Rezolvarea problemei P2

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Remarci 3

- Din demonstrația de mai sus urmează că pentru a rezolva P2 este suficient să se găsească o soluție a sistemului de ecuații (B). Drumurile de cost minim corespunzătoare pot fi obținute ca la (a) din demonstrația de mai sus: dacă avem $u_i = u_k + a_{ki}$ atunci k este nodul dinaintea lui i pe drumul de cost minim de la s la i (de cost u_i). În algoritmul care rezolva sistemul (B) menținem un tablou $before[1..n]$ cu elemente din $V \cup \{0\}$ cu semnificația finală $before[i] = \text{nodul dinaintea lui } i \text{ pe un drum de cost minim de la } s \text{ la } i$. Nodurile de pe acest drum pot fi determinate în $\mathcal{O}(n)$ prin construcția secvenței $i, before[i], before[before[i]], \dots, s$.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Drumuri de cost minim - Rezolvarea problemei P2

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Remarci 5

- Dacă, în problemele P1 - P3, G este un graf și nu un digraf, putem folosi algoritmi pentru digrafuri înlocuind fiecare muchie a lui G cu o pereche simetrică de arce, fiecare cu același cost ca muchia. Această abordare funcționează doar pentru muchii de cost nenegativ (dacă o muchie are cost negativ, atunci 2-circuitul corespunzător format cu cele două arce simetrice are cost negativ, deci ipoteza (I') este încălcată).

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul de săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 1.

Spunem că un graf $G = (V, E)$ este **rar** dacă $m \leq cn^2/\log n$ ($n = |V|, m = |E|$). Motivul este acela că putem reprezenta matricea de adiacență A a lui G folosind doar $\mathcal{O}(n^2/\log n)$ spațiu de memorie așa încât răspunsul la întrebarea " $a(i, j) = 1?$ " să poată fi dat în $\mathcal{O}(1)$.

Descrieți o astfel de reprezentare.

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 2.

Arătați că nu există o ordonare $e_1, e_2, \dots, e_{10}$ a muchiilor unui graf K_5 , așa încât: e_{10} și e_1 nu sunt adiacente și e_i și e_{i+1} nu sunt adiacente pentru orice $1 \leq i \leq 9$.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 3. Fie $G = (V, E)$ un graf de ordin n și dimensiune m cu matricea de adiacență A . Din mulțimea celor 2^m orientări posibile ale tuturor muchiilor sale alegem una și considerăm matricea de incidență nod-arc $Q \in \mathcal{M}_{n \times m}(\{-1, 0, 1\})$.

$$q_{ve} = \begin{cases} -1, & \text{dacă } v \text{ este extremitatea inițială a arcului } e \\ 1, & \text{dacă } v \text{ este extremitatea finală a arcului } e \\ 0, & \text{dacă } e \text{ nu este incident cu } v. \end{cases}$$

Arătați că $A + QQ^T$ este matrice diagonală și aflați interpretarea combinatorială a elementelor sale diagonale.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exercițiul 4. Fie $D = (V, E)$ un digraf cu $V = \{v_1, v_2, \dots, v_n\}$ și $E = \{e_1, e_2, \dots, e_m\}$. Fie $B = (b_{ij}) \in \mathcal{M}_{n \times m}(\{-1, 0, 1\})$ matricea de incidență a lui D , unde

$$b_{ij} = \begin{cases} 1, & \text{dacă } e_j \text{ este incident dinspre } v_i \\ -1, & \text{dacă } e_j \text{ este incident către } v_i \\ 0, & \text{altfel} \end{cases}.$$

Arătați că $\det(M) \in \{-1, 0, 1\}$ pentru orice submatrice pătratică, M , a lui B (i. e., B este o **matrice total unimodulară**).

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 5. Fie $G = (S, T; E)$ un graf bipartit cu $V = S \cup T = \{v_1, v_2, \dots, v_n\}$ și $E = \{e_1, e_2, \dots, e_m\}$. Fie $B = (b_{ij}) \in \mathcal{M}_{n \times m}(\{0, 1\})$ matricea de incidență a lui G , unde

$$b_{ij} = \begin{cases} 1, & \text{dacă } e_j \text{ este incidentă cu } v_i \\ 0, & \text{altfel} \end{cases}.$$

Arătați că $\det(M) \in \{-1, 0, 1\}$ pentru orice submatrice pătratică a lui B (i. e., B este o matrice total unimodulară).

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exercițiul 6. Arătați că un digraf are o singură ordonare topologică dacă și numai dacă are un drum Hamiltonian iar toate celelate arce sunt orientate înainte relativ la direcția de parcurgere a acestui drum.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 7. Fie M_G matricea de incidență muchie-nod a unui graf dat $G = (V, E)$, adică $M_G = (m_{ij})_{\substack{1 \leq i \leq m \\ 1 \leq j \leq n}}$, unde

$$V = \{v_1, v_2, \dots, v_n\}, E = \{e_1, e_2, \dots, e_m\}.$$

$$m_{ij} = \begin{cases} 1 & \text{dacă } e_i \text{ este incidentă cu } v_j \\ 0 & \text{altfel} \end{cases}$$

- (a) Arătați că dacă T este un arbore, atunci prin îndepărtarea din M_T a unei coloane corespunzătoare unui nod dat se obține o matrice pătratică nesară.
- (b) Arătați că dacă C este un circuit, atunci M_C este matrice nesară dacă și numai dacă C este impar.

Exercițiul 8. Fie $G = (V, E)$ un graf cu n noduri și m muchii. Considerăm următorul algoritm:

```
 $G' \leftarrow G;$   
while  $(\exists u \in V(G') \text{ a. } \hat{.} d_{G'}(u) < m/n)$  do  
     $G' \leftarrow G' - u;$   
return  $G';$ 
```

- (a) Determinați complexitatea timp a unei implementări eficiente a acestui algoritm.
- (b) Arătați că graful returnat, G' , este nenul (are și noduri, dar și muchii).
- (c) Arătați că orice graf conține un drum de lungime cel puțin m/n .

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exercițiul 9. **Diametrul** unui graf G este cea mai mare distanță dintre any două noduri din G . Două noduri formează o **pereche diametrală de noduri** dacă distanța dintre ele este cât diametrul. Arătați că următorul algoritm determină pereche diametrală de noduri într-un arbore dat T :

- 1 plecând dintr-un nod al lui T , parcurge *bfs* (Breadth First Search) arborele T ; fie u ultimul nod vizitat.
- 2 mai parcurge o data *bfs* arborele T plecând din nodul u ; fie v ultimul nod vizitat.
- 3 returnează perechea (u, v) .

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul de săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exercițiul 10. Arătați că o parcurgere DFS poate fi utilizată pentru a determina un circuit par într-un graf 3-regulat în $\mathcal{O}(n)$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 11.

- (a) Arătați că pentru un graf bipartit cu n noduri și m muchii avem $4m \leq n^2$.
- (b) Descrieți un algoritm de complexitate timp $\mathcal{O}(n + m)$ care să decidă dacă un graf dat (cu n noduri și m muchii) este complementul unui graf bipartit.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul de săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 12. Demonstrați că un graf G este bipartit dacă și numai dacă orice subgraf indus H al lui G satisface inegalitatea: $2\alpha(H) \geq |H|$.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Exercițiul 13. Fie $G = (S, T; E)$ un graf bipartit și $X \in \{S, T\}$. G se numește **X -lanț** dacă nodurile lui X pot fi ordonate: $x_1, x_2, \dots, x_k$ ($|X| = k$) astfel ca

$$N_G(x_1) \supseteq N_G(x_2) \supseteq \dots \supseteq N_G(x_k)$$

- (a) Arătați că G este S -lanț dacă și numai dacă este T -lanț.
- (b) Să presupunem că G (care este bipartit) are ordinul n , dimensiunea m și este reprezentat folosind liste de adiacență. Descrieți un algoritm de recunoaștere a unui S -lanț de complexitate timp $\mathcal{O}(n + m)$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul de săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 14. Fie G un graf; notăm cu $b(G)$ graful obținut din G prin inserarea simultană a câte unui nou nod pe fiecare muchie a lui G .

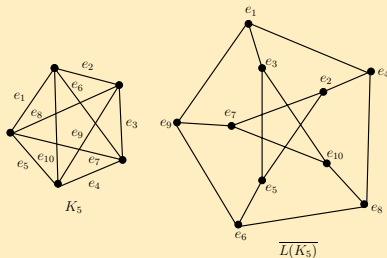
- Arătați că $b(G)$ este graf bipartit.
- Demonstrați că $G \simeq H$ dacă și numai dacă $b(G) \simeq b(H)$. De aici deduceți că testarea izomorfismului între două grafuri poate fi redusă în timp polinomial la testarea izomorfismului între două grafuri bipartite.

Exercițiul 15. Fie G un graf bipartit; arătați că G este conex dacă și numai dacă admite o singură bipartiție cu mulțimi stabile.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 2. Soluție.

- Să presupunem prin reducere la absurd că există o permutare a $E(K_5)$, $e_1, e_2, \dots, e_{10}$, așa încât oricare două muchii consecutive (dar și e_1 și e_{10} also) nu sunt adiacente.
- Astfel complementul grafului reprezentativ al muchiilor lui K_5 , $\overline{L(K_5)}$, are un circuit hamiltonian.
- $L(K_5)$ are 10 noduri și este 6-regulat (de ce?), deci $\overline{L(K_5)}$ este 3-regulat (vezi figura).



Exerciții rezolvate (parțial)

- Un graf hamiltonian și 3-regulat admite o 3-colorare a muchiilor.
- Cum se obține o astfel de colorare?
- Dar $\overline{L(K_5)}$ este graful lui Petersen (**verificați!**) care nu admite o astfel de colorare (pentru o foarte scurtă demonstrație: <https://www.sciencedirect.com/science/article/pii/S0012365X03001389>).

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.
Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 3. Soluție. Fie

$$B = A + QQ^T = (b_{uv})_{u,v \in V(G)},$$

unde $\vec{G}$ este orientarea aleasă a lui G . Fie u și v două noduri ale lui G .

$$b_{uv} = a_{uv} + \sum_{e \in E(\vec{G})} q_{ue} q_{ve}$$

- când $u \neq v$ și $uv \notin E(G)$ ($a_{uv} = 0$), pentru orice arc $e \in E(\vec{G})$, produsul $q_{ue} q_{ve} = ?$
- când $u \neq v$ și $uv \in E(G)$ ($a_{uv} = 1$ și există un singur arc, e_0 , incident cu amândouă nodurile u și v), pentru orice arc $e \in E(\vec{G})$, $e \neq e_0$, produsul $q_{ue} q_{ve} = ?$ iar produsul $q_{ue_0} q_{ve_0} = -1$. De ce?

Exerciții rezolvate (parțial)

- când $u = v$ avem $b_{uu} = \sum_{e \in E(\vec{G})} q_{ue}^2$ și

$$q_{ue}^2 = \begin{cases} 1, & \text{dacă } e \text{ este incident cu } u \\ 0, & \text{altfel} \end{cases}$$

- astfel $b_{uu} =$ numărul de arce incidente cu $u = ?$ (în termenii numărului de muchii incidente cu u în G).

Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exercițiul 4. Soluție.

- Pe fiecare coloană a lui B avem o valoare de 1, o valoare de -1 și 0 în rest.
- Fie M o submatrice pătratică a lui B .
- Demonstrăm prin inducție după k ordinul matricii M . Pentru $k = 1$, M este un element al lui B , deci $\det(M) \in \{-1, 0, 1\}$.
- În pasul inductiv presupunem că, $\det(M) \in \{-1, 0, 1\}$, pentru orice submatrice $k \times k$, M , a lui B ; fie M o submatrice $(k+1) \times (k+1)$. Sunt trei cazuri:

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

1. o întreagă coloană a lui M este nulă, astfel $\det(M) = ?$;
2. există o coloană h care conține un singur element nenul: $m_{ih} \in \{1, -1\}$, atunci

$$\det(M) = \sum_{j=1}^{k+1} m_{jh} (-1)^{j+h} \cdot \det(M_{jh}) = m_{ih} (-1)^{i+h} \cdot \det(M_{ih}),$$

unde $\det M_{ih}$ este minorul matricii B obținut prin ștergerea liniei i și a coloanei h din matricea B , astfel, din ipoteza inductivă $\det(M_{ih}) \in \{-1, 0, 1\}$ și $\det(M) \in \{-1, 0, 1\}$;

3. pe fiecare coloană a lui M avem un 1 și un -1 (restul elementelor rămase fiind 0); adunând toate liniile la una singură obținem o linie nulă (liniile lui M sunt liniar dependente). Astfel $\det(M) = ?$

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

Exercițiul 5. Soluție: similară cu cea de mai sus cu excepția faptului că matricea B nu este neapărat pătratică iar fiecare coloană a sa conține exact două valori nenule (egale cu 1) ceea ce modifică demonstrația ultimului caz de mai sus (**cum?**).

Exercițiul 6. Soluție. Fie $D = (V, E)$ un digraf; următorul algoritm construiește o ordonare aciclică a nodurilor lui D (dacă o astfel de ordonare există):

```
 $i \leftarrow 0;$   
while( $\exists x \in V$  with  $d_D^-(x) = 0$ ) {  
     $i++;$   
     $v_i = x;$   
     $V \leftarrow V \setminus \{x\};$   
    for ( $v \in \mathcal{A}^+(x)$ )  
         $d_D^-(v) --;$   
}
```

- Demonstrația se bazează pe următorul fapt: într-un digraf aciclic există noduri de grad interior 0 (dar și noduri de grad exterior 0).

De ce?

Exerciții rezolvate (parțial)

- “ $\implies$ ” Dacă D are o singură ordonare aciclică, atunci nu conține circuite iar în bucla *while* de mai sus la orice iterație există un singur nod de grad interior 0. De ce?
- La iterația i , v_{i+1} are gradul interior 1, astfel $v_i v_{i+1} \in E$, $\forall i = \overline{1, n-1}$; obținem de aici drumul hamiltonian dorit.
“ $\longleftarrow$ ” Dacă $v_1, v_2, \dots, v_n$ nodurile unui drum hamiltonian (în această ordine), atunci algoritmul de mai sus va construi aceeași aceeași ordonare aciclică. De ce?

Exercițiul 7. Soluție.

- (a) Ordonăm nodurile (coloanele) de la frunze spre rădăcină (putem transforma arborele într-unul cu rădăcină printr-o parcurgere **bfs**) nivel cu nivel. Ștergem coloana corespunzătoare rădăcinii.
- Definim următoarea ordonare a muchiilor (liniilor) și nodurilor (coloanelor): prima coloană corespunde unei frunze iar prima linie muchiei incidente cu acea frunză, a doua coloană corespunde unei frunze din arborele obținut prin ștergerea frunzei anterioare iar a doua linie muchiei incidente cu acea frunză etc.
 - Matricea rezultată este superior triunghiulară și are doar valori de 1 pe diagonală. **De ce?**

Exerciții rezolvate (parțial)

(b) Fie $V(C) = \{x_1, x_2, \dots, x_{2p+1}\}$ și $E(C) = \{x_1 x_2, x_2 x_3, \dots, x_{2p+1} x_1\}$.

- **Descrieți M_C și calculați-i determinantul.**

Exercițiul 8. Soluție. (a)

```
 $G' \leftarrow G;$   
calculează  $(d_{G'})$  pentru orice  $u \in V(G)$ ; //  $\mathcal{O}(n + m)$ , de ce?  
for  $(k = \overline{1, n - 1})$  do  
     $B_k \leftarrow \emptyset$ ; //  $\mathcal{O}(n)$ ;  
for  $(u \in V(G))$  do  
     $mark[u] \leftarrow 1$ ;  $push(B_{d_G(u)}, u)$ ; //  $\mathcal{O}(n)$ , ca la BucketSort, de ce?  
let  $d = \delta(G)$ ; //  $\mathcal{O}(n)$ ;  
while  $(d < m/n)$  do  
     $u \leftarrow pop(B_d)$ ;  $mark[u] = 1$ ; //  $\mathcal{O}(1)$ ;  
    for  $(x \in A_{G'}(u))$  do  
        if  $(mark[x] = 0)$  then  
             $push(B_{d_G(x)-1}, x)$ ; //  $\mathcal{O}(1)$ ;  
             $delete(A_{G'}(x), u)$ ; //  $\mathcal{O}(1)$ , de ce?  
    let  $d = \min\{h : B_h \neq \emptyset\}$ ; //  $\mathcal{O}(1)$ ;  
return  $G'$ ; // prin listele de adiacență;
```

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

(b) La fiecare pas al algoritmului ștergem un nod și mai puțin de m/n muchii (de ce?), deci numărul total de muchii eliminate este strict mai mic decât $n \cdot m/n = m$. Astfel G' nu poate fi nul.

(c) $G' \subseteq G$ și orice nod u din G' are $d_{G'}(u) \geq m/n$.

```
let  $x \in V(G')$ ;  $V(P) \leftarrow \{x\}$ ;  $E(P) \leftarrow \emptyset$ ;
```

```
while ( $N_G(x) \neq \emptyset$ ) do
```

```
  let  $u \leftarrow pop(N_G(x))$ ;
```

```
   $V(P) \leftarrow V(P) \cup \{x\}$ ;
```

```
   $E(P) \leftarrow E(P) \cup \{xu\}$ ;
```

```
   $x \leftarrow u$ ;
```

Drumul construit în acest fel are lungimea cel puțin m/n . De ce?

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 11. Soluție.

(a) Numărul de muchii dintr-un graf bipartit este egal cu suma gradelor nodurilor dintr-o clasă a bipartiției. **De ce?**

- Dacă $G = (S, T; E)$ is a bipartite graph:

$$\begin{aligned} m = |E(G)| &= \sum_{v \in S} d_G(v) \leq \sum_{v \in S} |T| = |S| \cdot |T| \stackrel{\text{de ce?}}{\leq} \\ &\leq \frac{(|S| + |T|)^2}{4} = ? \end{aligned}$$

(b) Dacă $\overline{G}$ este un graf bipartit, atunci

$$m = |E(G)| = \frac{n(n-1)}{2} - |E(\overline{G})| \geq \frac{n(n-1)}{2} - \frac{n^2}{4} = \frac{n(n-2)}{4}$$

- Dacă această inegalitate nu are loc, atunci complementul lui G nu poate fi bipartit.

- Altfel, dimensiunea lui G , $m = \mathcal{O}(?)$ ceea ce înseamnă că $\mathcal{O}(n + m) = \mathcal{O}(?)$.
- Putem construi listele de adiacență ale lui $\overline{G}$ din cele ale lui G în $\mathcal{O}(n + m)$. Putem aplica apoi un algoritm simplu de recunoaștere a grafurilor bipartite asupra lui $\overline{G}$ în $\mathcal{O}(|V(\overline{G})| + |E(\overline{G})|) = \mathcal{O}(?)$. **De ce?**
- Algoritmul:
 $\text{bip} \leftarrow \text{true};$
 if $(4m < n(n - 2))$
 $\text{bip} \leftarrow \text{false};$
 else {
 construiește listele de adiacență ale lui $\overline{G}$;
 $\text{bip} \leftarrow \text{bipartiteness}(\overline{G});$
 }

Exercițiul 12. Soluție.

- “ $\implies$ ” Dacă H este un subgraf al unui graf bipartit, atunci H este bipartit (de ce?); $H = (S, T; E)$ și $|H| = |S| + |T| \leq 2\alpha(H)$ (de ce?).
- “ $\Leftarrow$ ” Să presupunem G nu este bipartit, atunci G conține un circuit impar (nu neapărat indus): C cu $|V(C)| = 2k + 1$.
- Fie $H = [V(C)]_G$ subgraful indus de nodurile lui C . Din ipoteza știm că $\alpha(H) \geq \frac{1}{2}|H|$; dacă ștergem din H toate muchiile care nu sunt în C , obținem graful C_{2k+1} , astfel $\alpha(C_{2k+1}) \geq \alpha(H) \geq \frac{2k+1}{2}$.
- Urmează că $\alpha(C_{2k+1}) \geq \alpha(H) \geq k + 1$. Dar $\alpha(C_{2k+1}) = k$ (de ce?) - contradicție.

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 13. Soluție.

- (a) Fie $G = (S, T; E)$ un graf bipartit care este și S -lanț; presupunem prin reducere la absurd că G nu ar fi T -lanț, atunci există două noduri $u, v \in T$ astfel ca $N_G(u) \not\subseteq N_G(v)$ și $N_G(v) \not\subseteq N_G(u)$ (de ce?);
- Putem alege $x \in N_G(u) \setminus N_G(v)$ și $y \in N_G(v) \setminus N_G(u)$; urmează că $u \in N_G(x) \setminus N_G(y)$ și $v \in N_G(y) \setminus N_G(x)$;
 - Cum G este un S -lanț, $(x, y \in S) N_G(x) \subseteq N_G(y)$ sau $N_G(y) \subseteq N_G(x)$ - contradicție.

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- (b) Sortăm nodurile lui S în ordinea descrescătoare a gradelor (în $\mathcal{O}(n)$)
- de ce? - folosind bucketSort sau radixSort):

$$d_G(x_1) \geq d_G(x_2) \geq \dots \geq d_G(x_k)$$

- Apoi aplicăm următoarea procedură (de complexitate timp $\mathcal{O}(n + m)$), folosind observația că dacă G este S -lanț și $u, v \in S$ cu $d_G(u) \geq d_G(v)$, atunci trebuie să avem $N_G(u) \supseteq N_G(v)$ (de ce?):

Observație: pentru a face demonstrația independentă de alegerea bi-partiției, se poate demonstra că G este S -lanț dacă și numai dacă este $2K_2$ -free. Cum?

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

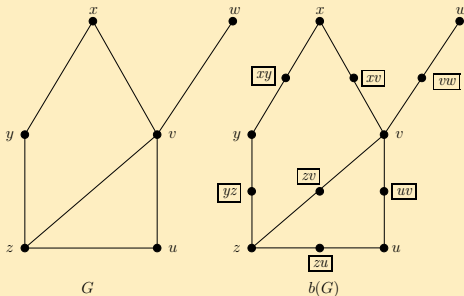
```
chain  $\leftarrow$  true;
for  $u \in V$ 
    neighbour[ $u$ ]  $\leftarrow$  0;
for  $u \in \mathcal{A}(x_1)$ 
    neighbour[ $u$ ]  $\leftarrow$  1;
for  $i = 2$  to  $k$ 
    for  $u \in \mathcal{A}(x_i)$ 
        if (neighbour[ $u$ ]  $\neq i - 1$ ) {
            chain  $\leftarrow$  false;
            return;
        }
    else
        neighbour[ $u$ ]  $\leftarrow$   $i$ ;
```

Dacă `chain` rămâne `true`, atunci $N_G(x_1) \supseteq N_G(x_2) \supseteq \dots \supseteq N_G(x_k)$,
altfel, folosind observația de mai sus G nu poate fi S -lanț. **De ce?**

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 14. Soluție. Fie $G = (V, E)$; notăm cu $\boxed{uv}$ noul nod inserat pe muchia uv (vezi mai jos).



Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

- Fie $\widehat{V} = \{\boxed{uv} : uv \in E(G)\}$; atunci $V(b(G)) = V \cup \widehat{V}$ și $E(b(G)) = \bigcup_{uv \in E} \{u\boxed{uv}, v\boxed{uv}\}$.

(a) $b(G)$ este un graf bipartit cu bipartiția $(V, \widehat{V})$: V și $\widehat{V}$ sunt mulțimi stabile în $b(G)$. De ce?